



AI with Rav



MACHINE LEARNING

with Rav

DAY 3 · VIDEO 3

From raw data to a working model, step by step

The ML Workflow — how a model is really built

In One Line: A real ML model isn't just "train it." It's a 7-step recipe — collect, clean, split, choose, train, evaluate, deploy. Skip a step and it breaks.

Start here: cooking biryani

You don't just throw rice in a pot and get biryani. There's a **recipe**: buy ingredients, wash the rice, marinate, layer, cook, taste, serve.

Machine Learning is exactly the same. Beginners think ML = "train the model." But `model.fit()` is just **one step out of seven**. The magic is in the whole recipe — and most of the work happens *before* training.

The full workflow, one picture



The 7 real steps. Notice the loop: if the model isn't good, you go back and try again. ML is iterative.

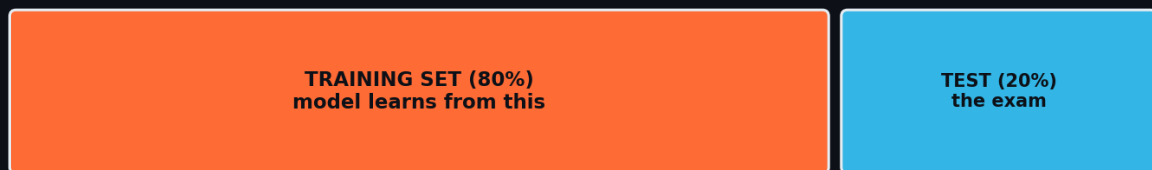
Step 1-2: Collect & Clean (80% of the real work)

- **Collect** → gather examples. For chai sales: 1000 days of (weather, day, festival → cups sold).
- **Clean** → real data is messy. Missing values, typos, wrong units, outliers. You fix all of it.

Common shock: in real jobs, ML engineers spend ~80% of time here — collecting and cleaning data — NOT on fancy algorithms. Garbage in = garbage out. Clean data beats a clever model every time.

Step 3: Split — the golden rule

Train / Test Split — the golden rule



Never let the model see the test set while learning — that would be cheating.

Split your data: most of it to LEARN from, a held-back piece to TEST honestly.

You **hide** some data (the test set) and let the model learn only from the rest. Then you check it on the hidden data — data it has never seen.

Never let the model see the test data while learning. That's like giving a student the exam paper beforehand — of course they'll "pass", but they learned nothing. This mistake is called **data leakage**.

Step 4-5: Choose a model & Train

- **Choose** → pick the algorithm that fits the problem (Linear Regression for prices, Decision Tree for yes/no, etc. — we cover these next).
- **Train** → the famous `model.fit(X_train, y_train)`. The model adjusts itself until its guesses match the answers.

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LinearRegression()
model.fit(X_train, y_train)           # step 5: learn from TRAIN only
```

Step 6: Evaluate — the honest exam

Now test on the hidden data. Does it predict well on data it never saw? If yes → great. If no → **go back** (the loop in the diagram): get more data, clean better, or pick a different model.

```
score = model.score(X_test, y_test)    # how good on UNSEEN data?
print(score)    # e.g. 0.89    -> 89% – decent, ship it
```

This is the moment of truth. A model that scores 99% on training data but 60% on test data is **memorising, not learning** (we'll name this "overfitting" in Video 16).

Step 7: Deploy

A model on your laptop is worth **■0**. Deploy = put it behind an API so a real app can send new data and get predictions. That's when ML earns money.

Recap — the 20-second version

- ML is a 7-step recipe, not just "train the model".
- Collect → Clean → Split → Choose → Train → Evaluate → Deploy.
- **80% of real work is data** (collect + clean), not algorithms.
- **Golden rule:** never let the model see test data while learning (data leakage).
- It's a **loop** — if evaluation is bad, go back and improve.

Next up — Video 4: Data — the fuel of ML. Features, labels, and why "garbage in, garbage out" decides everything. See you Day 4.